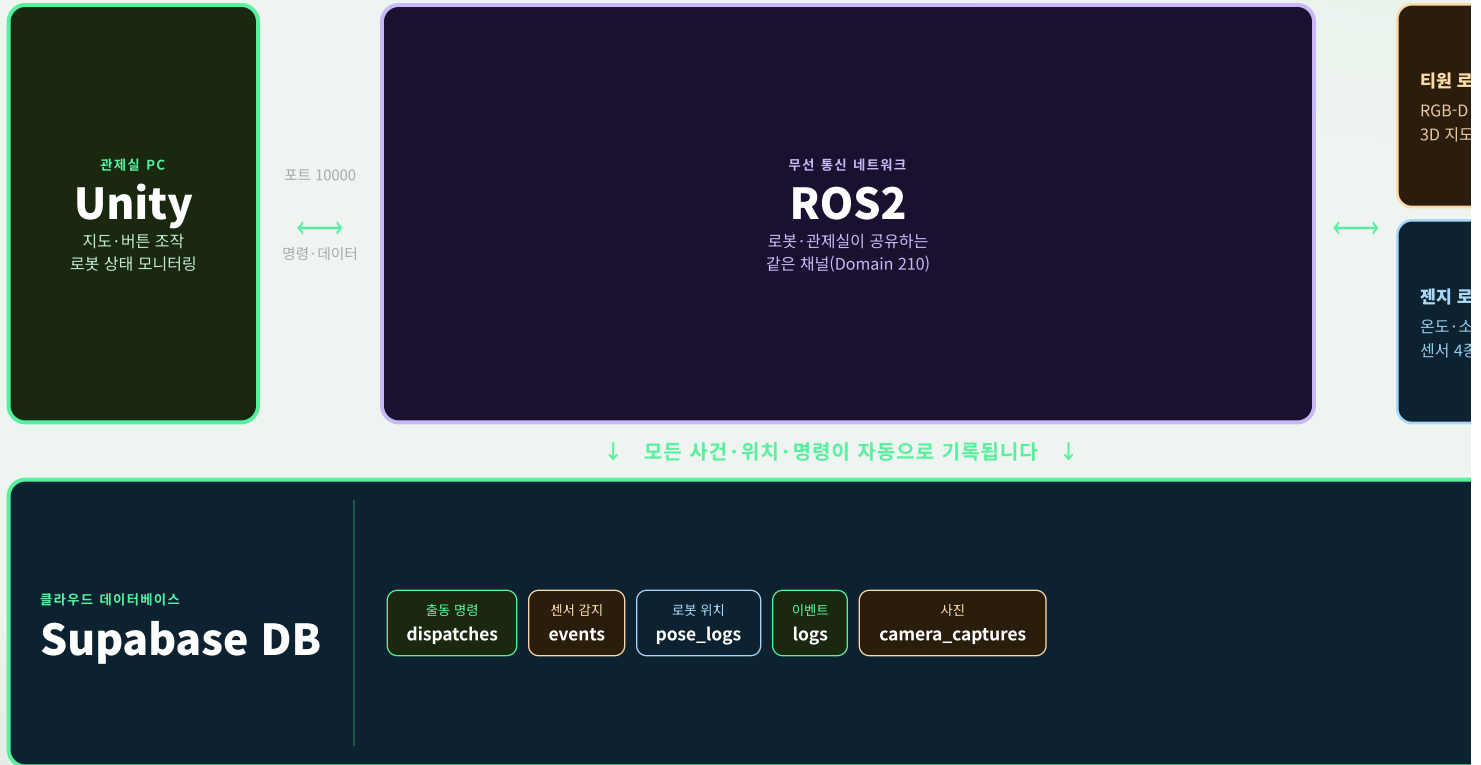


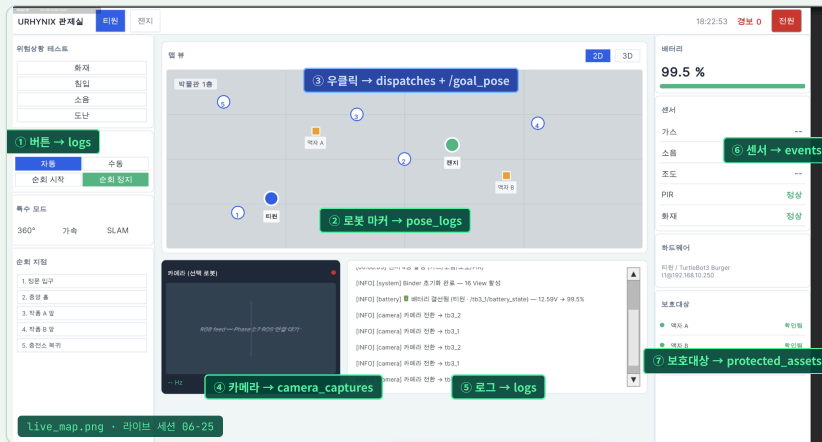
01 · 소프트웨어 아키텍처

클릭 → 명령 전달 → 로봇 출발 → 기록 저장 → 화면 갱신



02 · Unity 관제실 — 전체 화면

화면 각 구역이 어떤 DB 테이블로 연결되나



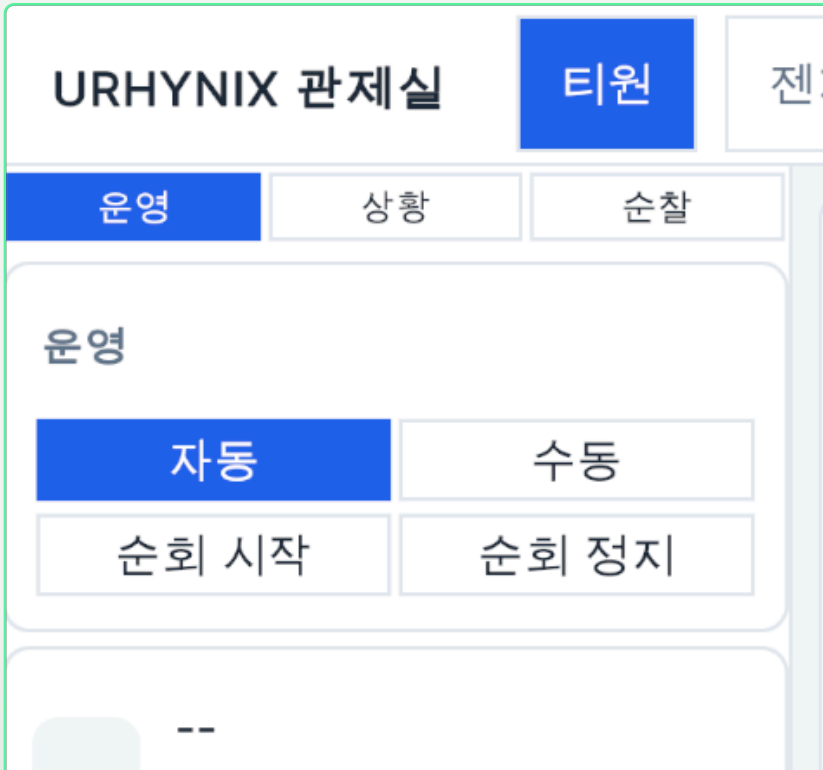
화면 구역	DB 테이블	설명
① 왼쪽 버튼 패널	logs	화재·침입·소음·도난 클릭 시 기록
② 지도 위 로봇 마커	pose_logs	1초마다 로봇 좌표(x,y,방향) 저장
③ 지도 우클릭 출동	dispatches	명령 저장 + ROS로 로봇에 목표 전송
④ 카메라 패널	camera_captures	로봇 PC가 직접 저장 (Unity는 보지 않음)
⑤ 이벤트 로그	logs	모든 UI 동작 자동 기록
⑥ 센서 카드	events	Arduino → ROS → 로봇 PC가 저장
⑦ 보호대상 목록	protected_assets	액자·중요품 등록 (개발 예정)

Unity가 직접 DB에 쓰는 것

session_meta(시작 1회) · dispatches(출동) · pose_logs(위치) · logs(로그)
 센서 · 카메라는 로봇 PC Python 스크립트가 직접 저장.

03 · 관제실 — 왼쪽 패널 확대

버튼 누르면 → logs 테이블에 한 줄 쌓인다



화재·침입·소음·도난 버튼

버튼을 누르면 "무슨 사건이 언제 발생했는지"
서버에 자동으로 기록이 남습니다

자동 / 수동 운영 모드

자동: 정해진 5개 지점을 혼자 계속 돌아다님
수동: 관리자가 지도를 클릭해 직접 이동 지시

순찰 지점 5개 등록

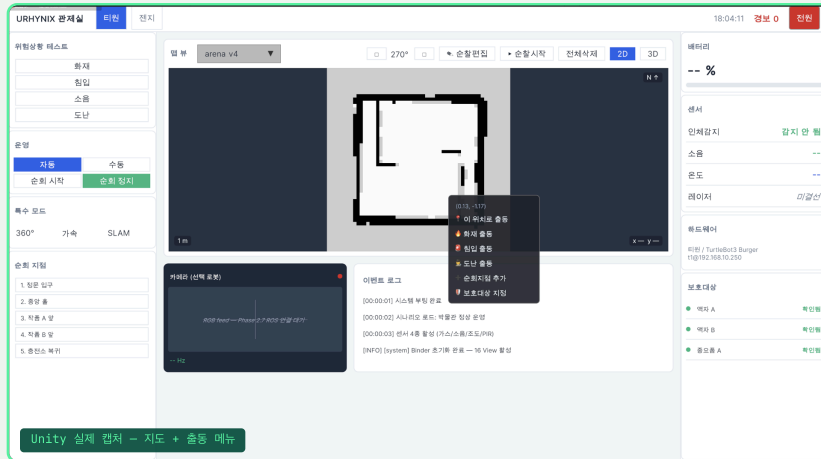
정문 입구 → 중앙 홀 → 작품 A앞 → 작품 B앞 → 충전소
자동 모드에서 이 순서를 반복합니다

Wi-Fi가 끊겨도 기록은 안 사라집니다

버튼 누르는 순간 화면은 즉시 반응하고,
서버 전송은 연결 복구 후 자동으로 재시도합니다.

04 · 관제실 — 지도 화면

지도 우클릭 → 출동 명령 메뉴 즉시 팝업



미리 만들어 둔 공간 지도

로봇이 촬영해 만든 지도가 화면에 표시됩니다
로봇 아이콘이 실시간으로 위치를 나타내며,
1초마다 좌표를 서버에 저장합니다

지도 우클릭 → 명령 메뉴 6가지

이 위치로 출동 / 화재·침입·도난 출동 선택
순찰지점 추가 / 보호대상 지정
선택하면 로봇이 즉시 그 위치로 이동합니다

순찰 지점 5개 등록

정문 입구·중앙 홀·작품 A앞·작품 B앞·출전소
자동 모드에서 이 경로를 계속 반복합니다

05 · 관제실 — 오른쪽 패널

센서·배터리·보호대상이 실시간 수치로 표시된다

배터리 잔량 — 현재 36.7%

로봇이 배터리 상태를 무선으로 전송하고
관제실 화면에 실시간 표시됩니다

센서 5종 수치 실시간 표시

가스 / 소음 / 조도 50% · 보통 / PIR 감지 안됨 / 화재 정상
센서 → 로봇 PC → 서버에 저장,
관제실은 서버에서 가져와 화면에 표시

연결된 로봇 정보

젠지 로봇 / TurtleBot3 Burger 모델
IP: 192.168.10.87

보호 대상을 3개 확인됨

액자 A · 액자 B · 중요품 A — 모두 **정상 확인**
서버 DB에 등록, 이상 시 경보 발령

배터리

36.7 %

센서

가스

소음

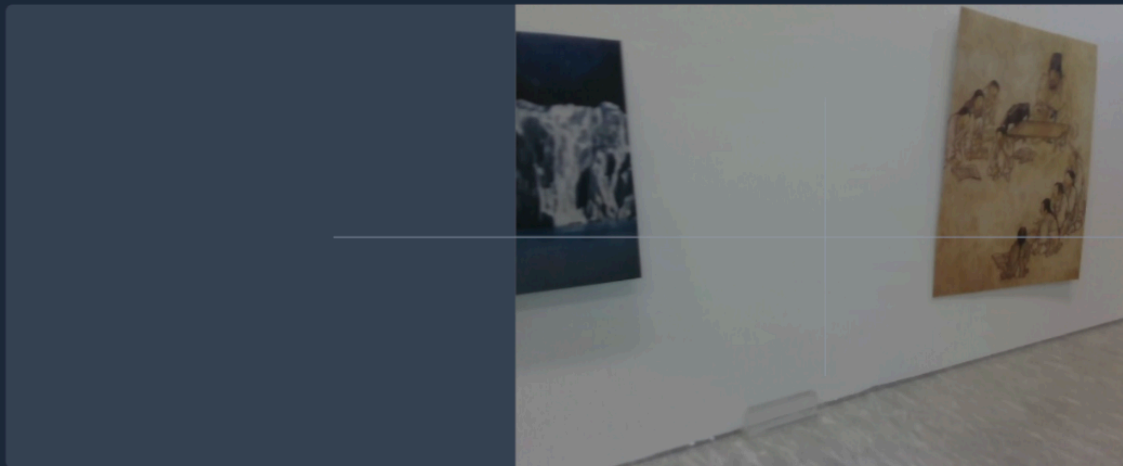
조도

50%

06 · 관제실 — 카메라 피드

로봇 카메라가 전시 작품을 실시간 촬영한다

카메라 (선택 로봇)

Unity 실제 캡처 — 14.9 Hz 카메라 피드 (작동 중)
14.9 Hz

로봇 눈 — 초당 15장 전송 중

로봇 카메라 영상이 실시간으로 관제실에 표시됩니다

로봇을 전환하면 그 로봇 카메라로 자동 전환

촬영 사진은 서버에 자동 저장

이벤트 로그 — 실시간 기록

시스템 부팅 완료

시나리오 로드: 박물관 정상 운영

센서 4종 활성화 (가스·소음·조도·PIR)

모든 줄이 서버 DB에 저장됩니다

07 · 출동 명령 흐름

지도 우클릭 한 번 → ROS + DB 동시 처리

① 관제실 화면

지도 우클릭

"이 위치로 출동"
선택

② 명령 생성

출동 요청 발생

어느 로봇을
어느 좌표로
보낼지 결정

③ 로봇에 전달

무선 명령 전송

로봇에게
"이 위치로 가라"
신호 발송

④ 기록 저장

서버 DB 저장

현재 - 어디로 -
왜 출동했는지
자동 기록

⑤ 로봇 출발

자율 주행 시작

로봇이 명령을
받고 목적지로
스스로 이동

③과 ④는 동시에 진행됩니다

로봇에 명령 전달과 DB 기록이 동시에 이루어집니다.
서버가 느려도 로봇은 즉시 출발합니다.

Wi-Fi가 끊겨도 기록은 안 사라집니다

인터넷이 끊기면 기록을 로컬에 임시 보관,
연결 복구 후 자동으로 서버에 다시 전송합니다.

08 · 데이터베이스 구조

하나의 세션 ID가 모든 기록을 하나로 묶는다

모든 기록의 출발점

session_meta

시연 1회 시작 → 세션 ID 1개 발급 → 아래 6개 테이블에 공유

dispatches

출동 명령 기록

언제 · 어디로 · 왜 출동했나
지도 클릭할 때마다 저장

events

센서 감지 기록

PIR·소음·온도·레이저
이상 감지 시 자동 저장

pose_logs

로봇 이동 경로

1초마다 로봇 좌표(x·y·방향)
정지 시에는 저장 생략

logs

화면 이벤트 로그

버튼 클릭 · 카메라 전환 등
화면의 모든 동작 기록

camera_captures

카메라 사진

로봇 PC가 촬영 · 저장
AI 이상감지 분석 예정

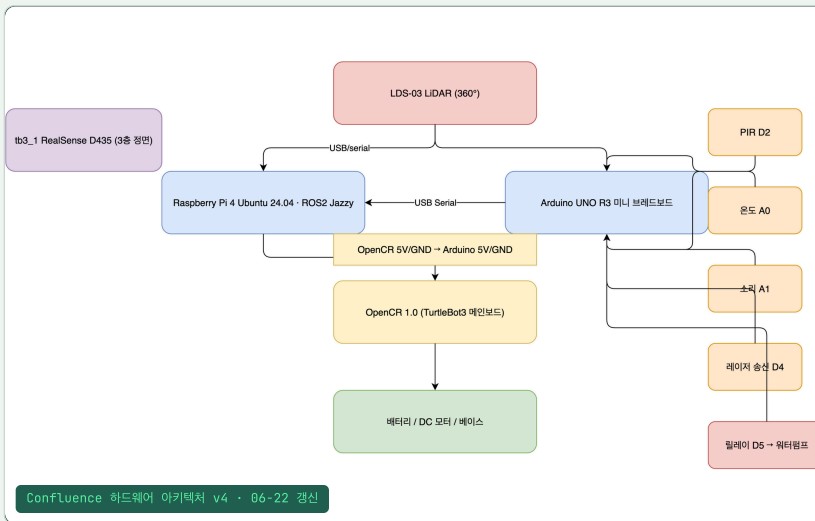
protected_assets

보호 대상물

액자 A·B·중요품 A 등록
상태 확인 · 경보 연동

09 · 하드웨어 구성

로봇 상판 위 · LiDAR + RPi + Arduino + 4센서



각 부품의 역할

LDS-03 라이다 — 360° 레이저로 주변 공간을 지도로 만들

라즈베리파이 — 로봇의 두뇌. 통신·명령 처리 담당

Arduino — 4개 센서를 읽어 라즈베리파이에 전달

OpenCR — 바퀴 모터를 직접 제어하는 보드

D435 카메라 (티원만) — 거리 측정 카메라, 3D 지도 촬영

데이터 흐름

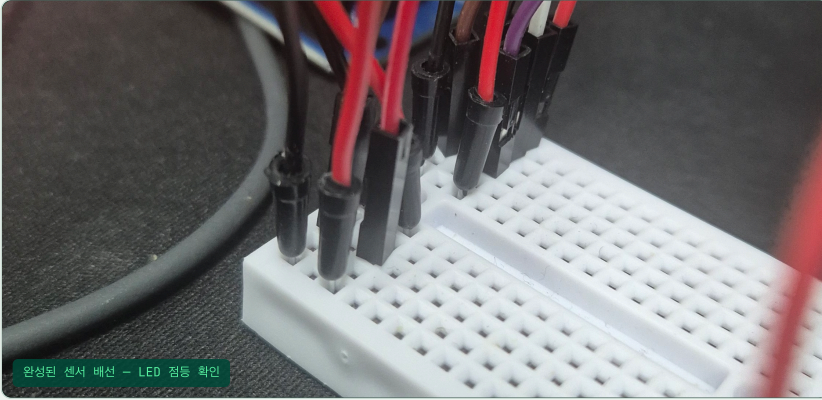
Arduino → (USB) → 라즈베리파이

라즈베리파이 → (Wi-Fi) → 관제실 PC / 서버

D435 카메라 → 라즈베리파이 → 녹화 파일 → Mac에서 3D 지도 생성

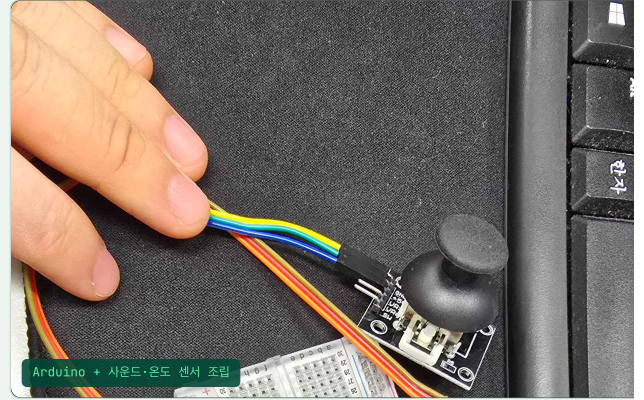
10 · 하드웨어 실물

직접 조립한 Arduino 센서 모듈



완성된 센서 배선 - LED 점등 확인

PIR(움직임) · 레이저(거리) · 소음 · 온도 센서
총 4개 센서를 Arduino에 연결해 직접 제작



Arduino + 사운드·온도 센서 조립

소음 센서 디지털 출력 문제 → 아날로그 값으로 대체 해결
USB 충돌 방지를 위해 사용 핀 변경

11 · RTAB-Map 이란?

D435 카메라로 3D 지도를 만드는 원리

RTAB-Map = Real-Time Appearance-Based Mapping

RGB-D 카메라(D435)로 찍은 **컬러 + 깊이** 영상을
3D 점들의 구름(Point Cloud)으로 변환하고,
로봇이 이동할수록 점들을 누적해 **3D 공간 지도**를 만든다.

일반 카메라와 D435의 차이

일반 카메라: 2D 이미지만 제공 (거리 모름)
D435: 픽셀마다 **거리(깊이)**를 함께 측정
→ 실제 3D 공간을 컴퓨터에 재현 가능

이번 주 해결한 핵심 문제

로봇에서 직접 3D 처리하는 방식이 작동하지 않아,
대안: 영상을 파일로 녹화 → Mac에서 3D 지도로 변환하는 방식 채택.

RTAB-Map 데이터 흐름

INPUT

D435 RGB 영상 + Depth 영상 + /odom 이동량



PROCESS

프레임마다 특징점 추출 → 같은 장소 재인식(루프 클로저) → 오차 자동 보정

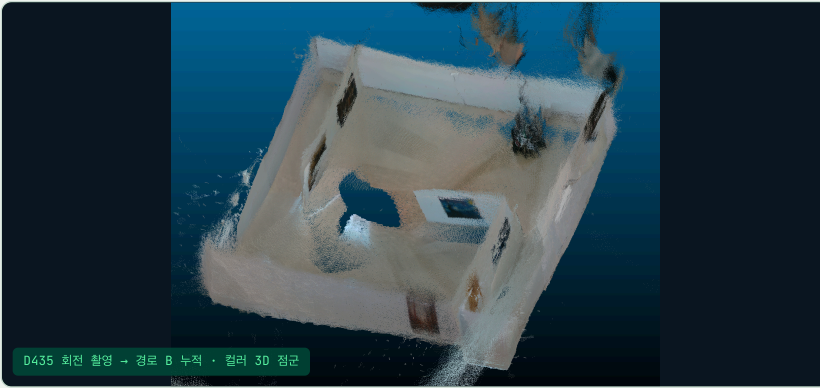


OUTPUT

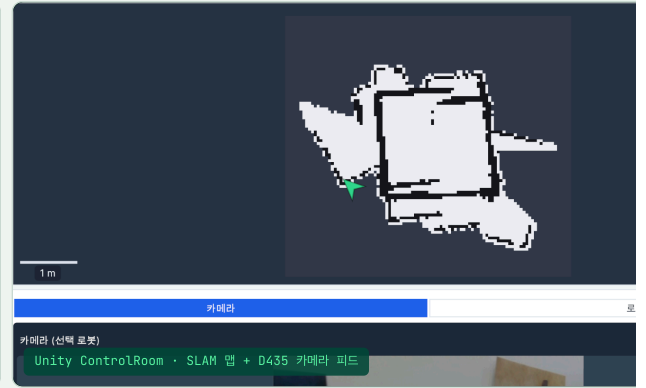
3D 점군 파일 — 207,069개 컬러 점으로 이루어진 공간 모델
지도 데이터베이스 — 79개 촬영 위치 기록 · 42MB

12 · RTAB-Map 결과

회전 한 번 → 컬러 3D 점군 207,069점



D435 회전 촬영 → 경로 B 누적 · 컬러 3D 점군



카메라 (선택 로봇)

Unity ControlRoom · SLAM 맵 + D435 카메라 피드

207K 3D 점 개수

점 207,069개가 모여 공간을 표현 · 3D 파일 + 42MB 지도 DB · 79개 촬영 위치 기록

5번 실패 후 성공

시계 불일치 → 압축 릴레이 → 통신 품질 → 환경 격리 순서로 원인 추적해 해결